



SDEV2301

INTRODUCTION TO ENTERPRISE APPLICATION DEVELOPMENT

Fall 2026

Introduction to Collection Queries

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

LINQ lesson plan

0–2 min · 2 minutes · [Open the provided starter solution](#)

TODAY'S GOALS

Read a lambda expression
Filter, project, and sort a sequence
Predict values and result types
Print results with foreach

OUR 110 MINUTES

0–50 Student queries and practice
50–60 Break
60–70 Sorting and coding
70–105 Pokémon in class
105–110 Exit ticket

LINQ = Language Integrated Query

Starter project ready to run

2–5 min · 3 minutes · [Continue Program.cs](#)

Open Lesson13_LinqIntro.slnx in Visual Studio

Confirm Student.cs, Pokemon.cs, and Program.cs

Build and run the supplied Console App

Students ready: 5

Pokémon ready: 20

Every coding task continues in Program.cs below the supplied data.

Warm-up: students, loops, and conditions

5–8 min · 3 minutes · Inspect Program.cs, then print the names

```
var students = new List<Student>
{
    new Student("Asha", 91),
    new Student("Chris", 68),
    new Student("Sofia", 77),
    new Student("Jordan", 84),
    new Student("Mei", 59)
};
```

Your 3-minute warm-up

Print every name with foreach.

Predict who has Mark ≥ 70 .

What does the loop variable represent on each pass?

Lambda-expression syntax

8–12 min · 4 minutes · Read the expression before using LINQ

A lambda is a small unnamed function passed to a method.

```
student => student.Mark >= 70
```

Part	Meaning
student	One input item (a Student here)
=>	Lambda operator: separates input from expression
student.Mark >= 70	Expression returns true or false (bool)

student => student.Name returns a string instead.

One goal in a loop and in LINQ

12–15 min · 3 minutes · Predict first, then reveal

```
var passingLoop = new List<Student>();  
foreach (var student in students)  
{  
    if (student.Mark >= 70)  
        passingLoop.Add(student);  
}
```

Predict the matching names · click to reveal the LINQ version

Same matching students. The result can be printed with foreach.

Where syntax: keep matching items

15–18 min · 3 minutes · Predicate = a test that returns bool

```
source.Where(item => trueOrFalseExpression)
```

```
IEnumerable<Student> passing =  
    students.Where(student => student.Mark >= 70);
```

Input item: Student Lambda result: bool

Query result: IEnumerable<Student>

Where changes which items remain. Each result item is still a Student.

Code: filter and print

18–25 min · 7 minutes · Continue Program.cs

1. Write a Where query for students with Mark $\geq$ 70.
2. Print the result. Use this loop after your query:

```
foreach (var student in passing)
{
    Console.WriteLine($"{student.Name}: {student.Mark}");
}
```

3. Change the condition to Mark $<$ 70. Predict, then run.

Comment the result type. Check students.Count is still 5.

IEnumerable<T>: a sequence of T

25–29 min · 4 minutes · Connect the new type to foreach

An interface is a contract: “I can provide items one at a time.”

Sequence type	Each item is a...
IEnumerable<Student>	Student
IEnumerable<string>	string
IEnumerable<int>	int

```
foreach (var student in passing)
    Console.WriteLine(student.Name);
```

foreach reads the sequence. A query result need not be a List<T>.

Select syntax: return a value per item

29–32 min · 3 minutes · Selector = the value to return

```
source.Select(item => valueExpression)
```

```
IEnumerable<string> names =  
    students.Select(student => student.Name);
```

Input item: Student Lambda result: string

Query result: `IEnumerable<string>`

Select returns one value for every input item. Here, all five names.

Code: names, marks, and labels

32–38 min · 6 minutes · Continue Program.cs

Write three Select queries and print each result:

```
names    // each student.Name  
marks    // each student.Mark  
labels   // $"{student.Name}: {student.Mark}"
```

Before running: write IEnumerable<...> beside each query.

Check: five results each. Explain why no student disappeared.

Where and Select have different jobs

38–41 min · 3 minutes · Write an answer, then compare with a partner

Question	Operation and result
Keep students with Mark ≥ 70	Prediction 1 · click to reveal
Return each student's Name	Prediction 2 · click to reveal
Return names of passing students	Prediction 3 · click to reveal
What happens to the source list?	Prediction 4 · click to reveal

Where and Select in one chain

41–44 min · 3 minutes · Follow the item type after each method

```
var passingNames = students
    .Where(student => student.Mark >= 70)
    .Select(student => student.Name);
```

Stage	Type
students	List<Student>
after Where	IEnumerable<Student>
after Select	IEnumerable<string>

After selecting Name, each item is a string and has no Mark property.

Code: student-query challenge

44–50 min · 6 minutes · Continue Program.cs

1. Names of students with Mark $\geq$ 80
2. Marks of students with Mark $<$ 70
3. Names with at least five characters

For each query: predict the values and result type, then run.
Use Where and Select. Use foreach to print the results.

Finished early: return passing students' names in uppercase.

10-minute break

50–60 min · Return at minute 60

Keep the starter solution open.

Next: sorting, then the Pokémon coding activity.

OrderBy and OrderByDescending

60–64 min · 4 minutes · The lambda chooses a sorting key

```
var lowToHigh = students.OrderBy(student => student.Mark);
```

```
var highToLow = students.OrderByDescending(student => student.Mark);
```

OrderBy: lowest first OrderByDescending: highest first

The key is an int. The result items are still Student objects.

Return type: `IOrderedEnumerable<Student>`

This is also an `IEnumerable<Student>`, so foreach still works.

Code: sort the students

64–70 min · 6 minutes · Continue Program.cs

1. Print Name and Mark from lowest mark to highest.
2. Reverse the order with OrderByDescending.
3. Return passing students' names, highest mark first.

```
students
    .Where(/* condition */)
    .OrderByDescending(/* key */)
    .Select(/* value */)
```

Check each stage's type.

Print students again.
Did its order change?

LINQ Basics: Pokémon in class

70–73 min · 3 minutes · Core activity · Continue Program.cs

[Open worksheet: LINQ Basics – Pokémon \(sections A–D\)](#)

The starter already contains Pokemon.cs and 20 Pokémon.

For every task

Write the query.

Predict its result type in a one-line comment.

Print the values with foreach and check them.

Work through sections A–D in class. Queries go below the supplied data.

Pokémon work block: filter and project

73–85 min · 12 minutes · Worksheet sections A and B

A · Complete the five filtering tasks

B · Complete the three projection tasks

At minute 79: compare one Where result and one Select result with a partner. Explain why their element types differ.

Stuck? Identify the input item, then decide what the lambda must return.

Pokémon work block: chain and explain

85–100 min · 15 minutes · Worksheet sections C and D

C · Complete the four filter-and-project tasks

D · Explain the three supplied queries in plain English

For each explanation, name what is kept, what is returned, and the sequence's element type.

Finished A–D? Return Pokémon names in descending Speed order.

Pokémon debrief: values and types

100–105 min · 5 minutes · Compare your code with a partner

```
var strongNames = pokedex
    .Where(p => p.Attack >= 120)
    .Select(p => p.Name);
```

Predict the type after Where · click to reveal

Predict the type after Select · click to reveal

Show one result you corrected after comparing a prediction with a run.

LINQ exit ticket

105–110 min · 5 minutes · Submit individually before leaving

1. Explain the parts of `p => p.Speed`.
2. Write a query for Water Pokémon names, fastest first.
Use `Where`, `OrderByDescending`, and `Select`.
3. State the result type after each method in your chain.

Add one question you still have about LINQ.